C PROGRAMMING | EXAM STUDY GUIDE

File Based I/O

Complete Theory, Syntax, Diagrams & Solved Programs

This guide covers:

- Concept of a File & Text Files in C
- The predefined `FILE` pointer (stdio.h)
- Members of the FILE structure & buffering
- High-level Disk I/O functions: fopen, putc, getc, fclose, fgets, fputs, feof
- Solved file-handling programs with output
- Quick revision tables & frequently asked exam questions

Designed for University Exams • Competitive Exams • Teacher Recruitment Tests (TRT/TET)

Table of Contents

1. Concept of a File

2. Text Files in C

3. Concept of a Predefined FILE Pointer and Its Definition in stdio.h

4. Meanings of Different Members of the Structure Representing FILE

5. Disk I/O Functions (High-Level / Standard File I/O)

5a. fopen() — Opening a File

5b. putc() — Writing a Character

5c. getc() — Reading a Character

5d. fclose() — Closing a File

5e. fgets() — Reading a String

5f. fputs() — Writing a String

5g. feof() — Detecting End of File

6. Simple File-Based Programs (Solved, with Output)

7. Quick Revision — Tables, Exam Questions & Debugging Tips

SECTION 1

Concept of a File

1.1 Definition of a File

DEFINITION

A **file** is a named collection of related data or information stored on a secondary (non-volatile) storage device such as a hard disk. In C programming, a file is used to permanently store data so that it can be retrieved and used later, even after the program that created it has terminated.

Every file on disk is identified by a unique **filename** and has associated properties such as its **path**, **size**, and **type**. A C program interacts with a file through a *stream* — a logical interface that connects the program to the physical file.

1.2 Need for File Handling

When a C program runs, all variables and data are stored in **primary memory (RAM)**, which is volatile — the data is lost as soon as the program terminates or the system is powered off. File handling solves this problem and provides several benefits:

Need	Explanation
Permanent Storage	Data stored in a file remains available even after the program ends or the computer restarts.
Large Volume of Data	Files can hold data far larger than what can conveniently be entered through the keyboard each time.
Data Sharing	A file created by one program can be read and used by another program.
Avoids Re-entry	Removes the need to re-type input data every time a program is run.
Portability	Data can be transferred between different systems by simply copying the file.

1.3 Types of Files

Files in C are classified into **Text Files** and **Binary Files** based on the format in which data is stored and represented on the disk (i.e., how the bytes are encoded/interpreted) — not based on their content, purpose, or size.

Type	Description	Example
Text Files	Store data as a sequence of readable ASCII/Unicode characters, organized into lines terminated by a newline character.	`notes.txt`, `data.csv`
Binary Files	Store data in the same binary format used internally by the computer (0s and 1s); not directly human-readable.	`program.exe`, `image.jpg`

Exam Tip

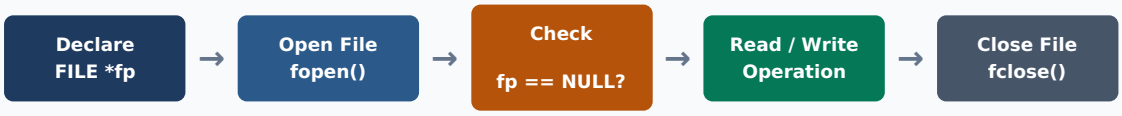
Text files are opened in **text mode** (`"r"` , `"w"` , `"a"`) while binary files use **binary mode** (`"rb"` , `"wb"` , `"ab"`). This distinction is a favourite viva/exam question.

1.4 Advantages of File Storage

- **Persistence:** Data survives beyond the lifetime of the program/process.
- **Capacity:** Secondary storage can hold far more data than RAM.
- **Reusability:** Stored data can be reused, updated, or shared across multiple programs.
- **Backup & Transport:** Files can be copied, backed up, or transferred across systems/networks.
- **Organized Access:** Data can be structured, indexed, and accessed selectively (sequential or random access).

1.5 Diagram: File Handling Process

FIG 1.1 — GENERAL FILE HANDLING PROCESS IN C



This five-step cycle — **declare pointer → open → validate → process → close** — is the backbone of every file-handling program in C and is repeated for each of the functions discussed in Section 5.

Section 1 — Key Takeaways

- A file is a named collection of data stored permanently on secondary storage.
- File handling overcomes the volatility and size limits of RAM.
- Files are classified as **text files** and **binary files**, based on their data storage/representation format.
- Main advantages: persistence, large capacity, reusability, and portability.

SECTION 2

Text Files in C

2.1 Definition

DEFINITION

A **text file** is a file that stores data as a sequence of characters organized into lines, where each line is terminated by a special **newline character** (`'\n'`). The content of a text file can be directly viewed and edited using a simple text editor.

2.2 Characteristics of Text Files

Characteristic	Details
Human Readable	Stored as ASCII/character codes, so contents can be read directly.
Line-Oriented	Data is organized into lines separated by `'\n'`.
Character Translation	On some systems, `'\n'` may be translated to a carriage-return/line-feed pair when written and translated back when read.
Slightly Larger Size	May take marginally more space than an equivalent binary file due to character-based storage of numbers.
Opened With	Mode strings such as `"r"`, `"w"`, `"a"`, `"r+"`, `"w+"`, `"a+"`.

2.3 Creating and Using Text Files

A text file is created in C by opening a file in write mode using `fopen()` with mode `"w"`. If the file does not exist, it is created; if it exists, its previous content is erased.

SYNTAX — CREATING A TEXT FILE

```
FILE *fp;
fp = fopen("data.txt", "w");   /* creates data.txt if absent */
```

2.3.1 Example — Writing to a Text File

PROGRAM

```
#include <stdio.h>
int main() {
    FILE *fp;
    fp = fopen("greet.txt", "w");     /* open in write mode */
    if (fp == NULL) {
        printf("File could not be opened\n");
        return 1;
    }
    fputs("Hello, File Handling in C!\n", fp);
    fclose(fp);
    printf("Data written successfully.\n");
    return 0;
}
```

OUTPUT

```
Data written successfully.

(A new file "greet.txt" is created on disk containing:
Hello, File Handling in C!)
```

2.3.2 Example — Reading from a Text File

PROGRAM

```
#include <stdio.h>
int main() {
    FILE *fp;
    char line[100];
    fp = fopen("greet.txt", "r");     /* open in read mode */
    if (fp == NULL) {
        printf("File not found\n");
        return 1;
    }
    while (fgets(line, sizeof(line), fp) != NULL)
        printf("%s", line);
```

```
    fclose(fp);
    return 0;
}
```

OUTPUT

```
Hello, File Handling in C!
```

Exam Tip

Always check whether `fopen()` returned `NULL` before using the file pointer — attempting file operations on a NULL pointer causes a runtime crash. This is one of the most frequently asked "identify the error" questions.

Simple Way to Remember

- `"r"` = **read only** — file must already exist, nothing is ever erased or created.
- `"w"` = **wipe and write** — always starts from a blank file (old content destroyed if it existed).
- `"a"` = **add at the end** — old content preserved, new content tacked on after it.

Section 2 — Key Takeaways

- Text files store human-readable, line-oriented character data.
- Created/opened using `fopen()` with text-mode strings ("r", "w", "a", etc.).
- Always validate the returned `FILE*` for `NULL` before use.

SECTION 3

Concept of a Predefined FILE Pointer and Its Definition in stdio.h

3.1 Definition of FILE

DEFINITION

`FILE` is a **predefined structure type** declared in the header file `<stdio.h>`. It is not a basic data type but a structure (commonly typedef-ed) that holds all the information the C library needs to control and manage a stream — such as its current position, buffering, and error/end-of-file status.

Programmers never need to know the exact internal layout of `FILE`; they simply declare a **pointer to FILE** and let library functions (`fopen`, `fclose`, etc.) manage the structure internally. This is a classic example of **data abstraction / information hiding**.

3.2 Declaration of a File Pointer

SYNTAX

```
FILE *fp;
```

Here, `fp` is declared as a pointer to an object of type `FILE`. This pointer will later be made to point to the FILE structure that `fopen()` creates and initializes for a particular disk file.

3.3 Purpose of FILE *

- It acts as a **handle / reference** that links every subsequent file operation (read, write, close) to the correct open file.
- It stores the **current read/write position** within the file (the file position indicator).
- It keeps track of **buffer** information used for efficient I/O.
- It records the **end-of-file** and **error** status of the stream.
- Every high-level I/O function (`fopen, fclose, fgetc, fputc, fgets, fputs, fread, fwrite, fprintf, fscanf` ...) requires a `FILE *` as a parameter to know which stream to operate on.

3.4 Example

PROGRAM

```
#include <stdio.h>
int main() {
    FILE *fp;                       /* declare file pointer */
    fp = fopen("record.txt", "w");  /* fp now points to the
                                       FILE structure for record.txt */
    if (fp != NULL) {
        fputs("File pointer example\n", fp);
        fclose(fp);
    }
    return 0;
}
```

OUTPUT

```
(No console output; "record.txt" is created containing:
File pointer example)
```

FIG 3.1 — FILE POINTER REPRESENTATION



fp does not store data itself — it stores the address of the FILE structure that the C library maintains for the opened stream.

Exam Tip

A common one-mark question: "*FILE is a* ____." Answer: **a structure type defined in stdio.h** (not a keyword, not a basic data type).

Section 3 — Key Takeaways

- `FILE` is a structure type defined in `<stdio.h>`.
- A **file pointer** (`FILE *fp`) is used to reference an opened stream.
- All standard file I/O functions operate through this pointer.

SECTION 4

Meanings of Different Members of the Structure Representing FILE

4.1 Internal Structure of FILE

Although the exact implementation of `FILE` differs across C library implementations (e.g., Turbo C vs GCC/glibc), conceptually every `FILE` structure maintains a common set of control fields. A typical (illustrative) definition looks like this:

ILLUSTRATIVE FILE STRUCTURE (CONCEPTUAL)

```
typedef struct {
    int    level;      /* fill/empty level of buffer     */
    unsigned flags;     /* File status flags               */
    char   fd;          /* File descriptor                 */
    unsigned char hold;  /* Ungetc char if no buffer        */
    int    bsize;       /* Buffer size                     */
    unsigned char *buffer; /* Data transfer buffer          */
    unsigned char *curp; /* Current active pointer          */
    unsigned istemp;    /* Temporary file indicator        */
    short  token;       /* Used for validity checking      */
} FILE;
```

Common Misconception

Students are NOT expected to memorise the exact fields above for every compiler — they vary by implementation. Exams typically ask for the **conceptual purpose** of each kind of member, which is covered in the table below.

4.2 Purpose of Its Members (Conceptual Explanation)

Member (typical name)	Conceptual Purpose
file descriptor (fd)	An integer that uniquely identifies the opened file to the operating system.
buffer	A pointer to the memory area used to temporarily hold data being transferred between the program and the disk (see buffering, below).
buffer size (bsize)	The size of the allocated buffer, which determines how much data is read/written from disk in one go.
current pointer (curp)	Points to the next character/byte position to be read or written within the buffer — i.e., the file position indicator.
level	Indicates how full or empty the buffer currently is, used internally to decide when to refill/flush it.
flags	A set of bits recording the stream's status — e.g., whether it is open for reading, writing, or has encountered an error.
error / EOF indicators	Flags checked internally by functions like `feof()` and `ferror()` to detect end-of-file or error conditions.
hold	A one-character holding area used by functions such as `ungetc()`.

4.3 File Buffering Concept

DEFINITION

Buffering is a technique in which data is temporarily held in an intermediate memory area (the **buffer**) before being transferred between the program and the disk, instead of performing a slow disk-access operation for every single character.

Because disk I/O is much slower than memory access, the C standard library reads/writes data from disk in large chunks into a buffer. Subsequent read/write calls are then served directly from this buffer in memory, drastically improving performance.

Buffering Type	Description
Fully Buffered	Data is transferred to/from disk only when the buffer is completely full (typical for disk files).
Line Buffered	Data is transferred whenever a newline character is encountered (typical for terminal/console I/O).
Unbuffered	Data is transferred immediately, character by character (e.g., `stderr`).

4.4 Diagram: FILE Structure Representation

FIG 4.1 — CONCEPTUAL FILE STRUCTURE

FILE structure (conceptual)

```
int    fd       // file descriptor
char *buffer    // pointer to buffer memory
int    bsize    // size of buffer
char *curp      // current position pointer
int    level    // fill level of buffer
unsigned flags  // status flags (EOF/error)
```

FIG 4.2 — DATA FLOW BETWEEN PROGRAM AND FILE (VIA BUFFER)



Data moves from program variables into the stream's buffer, and the buffer is flushed to (or filled from) the physical disk file — either automatically or via fflush()/fclose().

Section 4 — Key Takeaways

- The `FILE` structure internally stores a file descriptor, buffer pointer, buffer size, current position pointer, and status flags.
- Exact member names vary by compiler; exams focus on **conceptual purpose**, not exact syntax.
- **Buffering** improves I/O performance by reducing the number of slow disk accesses.
- Buffering types: fully buffered, line buffered, unbuffered.

SECTION 5

Disk I/O Functions (High-Level / Standard File I/O)

High-level (standard) file I/O functions are provided by `<stdio.h>` and operate through a `FILE *` stream, using internal buffering for efficiency. This section explains, in the syllabus order, the seven core functions: `fopen()`, `putc()`, `getc()`, `fclose()`, `fgets()`, `fputs()`, and `feof()`.

5(a) fopen() — Opening a File

PURPOSE

`fopen()` opens a specified file and associates a stream (a `FILE *`) with it, so that the file can be used for reading, writing, or appending.

SYNTAX

```
FILE *fopen(const char *filename, const char *mode);
```

Parameter	Meaning
`filename`	Name (and optional path) of the file to be opened.
`mode`	String specifying how the file should be opened (read/write/append, text/binary).
Return Value	A valid `FILE *` pointer on success, or `NULL` if the file could not be opened.

File Opening Modes

Mode	Meaning	If File Exists	If File Does Not Exist
`"r"`	Read only (text)	Opens for reading	Returns NULL (error)
`"w"`	Write only (text)	Erases existing content	Creates new file
`"a"`	Append (text)	Opens; writes go to the end	Creates new file
`"r+"`	Read and write	Opens for both	Returns NULL (error)
`"w+"`	Write and read	Erases existing content	Creates new file
`"a+"`	Append and read	Opens; writes appended at end	Creates new file
`"rb" / "wb" / "ab"`	Same as above but in **binary** mode	—	—

Simple Way to Remember

- `"r"` = **read only** — file must already exist, nothing is ever erased or created.
- `"w"` = **wipe and write** — always starts from a blank file (old content destroyed if it existed).
- `"a"` = **add at the end** — old content preserved, new content tacked on after it.

Working Principle

When `fopen()` is called, the C library asks the operating system to locate (or create) the named file and allocates a `FILE` structure in memory to represent it — setting up its buffer, file descriptor, and initial position indicator. It then returns the **address** of this structure to the calling program as a `FILE *`. All later operations use this pointer to reach the same structure, so the stream stays consistent from open to close.

Example

PROGRAM

```
#include <stdio.h>
int main() {
    FILE *fp;
    fp = fopen("test.txt", "w");
    if (fp == NULL) {
        printf("Error: could not open file\n");
        return 1;
    }
    printf("File opened successfully\n");
    fclose(fp);
    return 0;
}
```

OUTPUT

```
File opened successfully
```

Advantages	Limitations
• Single, uniform entry point for opening any text or binary file. • Automatically allocates and manages the internal buffer for the stream. • Supports multiple modes (read/write/append, text/binary) through one function.	• Fails silently (returns NULL) rather than raising an exception — the programmer must always check the return value. • Does not create missing intermediate directories in the path. • Mode string errors (e.g. typing "x" instead of "w") are not caught at compile time.

Common Error

Forgetting to check the return value of `fopen()` for `NULL`. Opening a non-existent file in `"r"` mode and then using the pointer directly causes undefined behaviour / a crash.

FIG 5.1 — FILE OPEN → PROCESS → CLOSE CYCLE



5(b) putc() — Writing a Character to a File

PURPOSE

`putc()` writes a single character to the specified output stream and advances the file position indicator by one.

SYNTAX

```
int putc(int char_val, FILE *fp);
```

Parameter	Meaning
`char_val`	The character to be written (passed/promoted as an `int`).
`fp`	Pointer to the output stream (file opened in write/append mode).
Return Value	The character written on success, or `EOF` on failure.

Working Principle

Internally, `putc()` converts its `int` argument to an `unsigned char` and places it into the stream's output buffer at the position marked by the current pointer. The buffer is transferred to disk only when it becomes full, when `fflush()` is called, or when the stream is closed — this is the practical application of the buffering concept explained in Section 4.3.

Example — Writing Characters to a File

PROGRAM

```
#include <stdio.h>
int main() {
    FILE *fp;
    char ch, str[] = "ABC";
    int i = 0;
    fp = fopen("chars.txt", "w");
    if (fp == NULL) return 1;

    while (str[i] != '\0') {
        putc(str[i], fp);   /* write one character at a time */
        i++;
    }
    fclose(fp);
    printf("Characters written to chars.txt\n");
    return 0;
}
```

OUTPUT

```
Characters written to chars.txt

(File "chars.txt" now contains: ABC)
```

Advantages	Limitations
• Very simple to use for fine-grained, character-level control over output. • Efficient for small, incremental writes due to internal buffering.	• Inefficient for writing large blocks of data compared to `fwrite()` or `fputs()`. • Requires an explicit loop to write strings, increasing code length.

Exam Tip

`putc()` and `fputc()` behave identically; `putc()` may be implemented as a macro, while `fputc()` is guaranteed to be a true function. Both are frequently treated as interchangeable in exam answers.

5(c) getc() — Reading a Character from a File

PURPOSE

`getc()` reads a single character from the specified input stream and advances the file position indicator by one.

SYNTAX

```
int getc(FILE *fp);
```

Parameter	Meaning
`fp`	Pointer to the input stream (file opened in read mode).
Return Value	The character read (as an `int`) on success, or `EOF` when the end of file is reached or an error occurs.

Working Principle

`getc()` fetches the next byte from the stream's input buffer (refilling the buffer from disk first if it is empty), returns it as an `int`, and advances the current-position pointer by one. When no more bytes remain, the underlying read fails and the function returns the special value `EOF`, which also sets the stream's end-of-file indicator (checked later by `feof()`).

Example — Reading Characters from a File

PROGRAM

```
#include <stdio.h>
int main() {
    FILE *fp;
    char ch;
    fp = fopen("chars.txt", "r");
    if (fp == NULL) return 1;

    while ((ch = getc(fp)) != EOF)
        printf("%c", ch);      /* read and print one char at a time */

    fclose(fp);
    return 0;
}
```

OUTPUT

```
ABC
```

Advantages	Limitations
• Simple and precise — allows character-by-character inspection or processing of file content. • Return type `int` lets it distinguish every possible byte value from `EOF`.	• Slower than block-based reads (`fread()`) for very large files. • Easy to misuse if the return value is stored in a `char` instead of an `int` (see Common Error below).

Common Error

Declaring the variable that stores `getc()`'s return value as `char` instead of `int`. Since `EOF` is typically `-1`, on systems where `char` is unsigned this comparison can fail, causing an infinite loop.

FIG 5.2 — CHARACTER I/O PROCESS (PUTC / GETC)



5(d) fclose() — Closing a File

PURPOSE

`fclose()` closes an open stream, flushing (writing out) any buffered data to disk and releasing all resources associated with the stream.

SYNTAX

```
int fclose(FILE *fp);
```

Parameter	Meaning
`fp`	Pointer to the stream to be closed.
Return Value	`0` on success, `EOF` on failure.

Example

PROGRAM

```
#include <stdio.h>
int main() {
    FILE *fp = fopen("log.txt", "w");
    if (fp == NULL) return 1;
    fputs("Closing example\n", fp);

    if (fclose(fp) == 0)
        printf("File closed successfully\n");
    else
        printf("Error closing file\n");
    return 0;
}
```

OUTPUT

```
File closed successfully
```

Working Principle

`fclose()` first flushes any data still sitting in the stream's output buffer to the physical disk file, then instructs the operating system to release the file descriptor and finally frees the memory used by the `FILE` structure itself. After this call, the pointer `fp` becomes invalid and must not be used again.

Advantages	Limitations
• Guarantees buffered data is safely written to disk before the program continues. • Frees system resources (file descriptors), allowing other files to be opened.	• Once closed, the same `FILE *` cannot be reused without calling `fopen()` again. • Does not automatically set the pointer to NULL, so a "dangling pointer" mistake is possible if reused accidentally.

Why Closing Matters

Data written using buffered functions may still be sitting in memory (the buffer) and not yet physically on disk. `fclose()` guarantees this data is **flushed** to disk. Forgetting to close a file can lead to **data loss** or **resource leaks** (running out of available file handles).

5(e) fgets() — Reading a String from a File

PURPOSE

`fgets()` reads a line (or up to a specified number of characters) from the input stream into a string buffer, stopping at a newline character or when the buffer limit is reached.

SYNTAX

```
char *fgets(char *str, int n, FILE *fp);
```

Parameter	Meaning
`str`	Pointer to the character array (buffer) where the read string will be stored.
`n`	Maximum number of characters to read (including the terminating null character) — i.e. at most `n-1` characters are read.
`fp`	Pointer to the input stream.
Return Value	`str` on success, or `NULL` on end-of-file / error.

Working Principle

`fgets()` repeatedly pulls characters from the stream's buffer into the destination array until it copies a newline character, copies `n-1` characters, or reaches end-of-file — whichever happens first. It then appends a terminating `'\0'` so the result is always a valid, null-terminated C string.

Example — Reading Lines from a File

PROGRAM

```
#include <stdio.h>
int main() {
    FILE *fp;
    char line[80];
    fp = fopen("poem.txt", "r");
    if (fp == NULL) return 1;

    while (fgets(line, sizeof(line), fp) != NULL)
        printf("%s", line);   /* line already includes '\n' */

    fclose(fp);
    return 0;
}
```

OUTPUT (ASSUMING POEM.TXT HAS TWO LINES)

```
Roses are red,
Violets are blue.
```

Advantages	Limitations
• Safer than `gets()` because it takes a size limit and cannot overflow the buffer. • Reads a full line in a single call, simplifying line-oriented processing.	• Stops early if the line is longer than `n-1` characters, splitting one logical line across multiple reads. • Retains the trailing `'\n'`, which the programmer must strip manually if not wanted.

Common Error

Unlike `gets()`, `fgets()` **retains** the newline character (`'\n'`) at the end of the string if it fits within the buffer. Forgetting this often causes unwanted blank lines when the string is later printed with an additional `"\n"`.

5(f) fputs() — Writing a String to a File

PURPOSE

`fputs()` writes a null-terminated string to the specified output stream, **without** automatically appending a newline character.

SYNTAX

```
int fputs(const char *str, FILE *fp);
```

Parameter	Meaning
`str`	Pointer to the null-terminated string to be written.
`fp`	Pointer to the output stream.
Return Value	A non-negative value on success, or `EOF` on failure.

Working Principle

`fputs()` copies characters from the source string into the stream's output buffer one after another, stopping as soon as it reaches the string's terminating `'\0'` — the null terminator itself is **not** written to the file. The buffer is flushed to disk according to the same buffering rules described in Section 4.3.

Example — Writing Strings to a File

PROGRAM

```
#include <stdio.h>
int main() {
    FILE *fp = fopen("poem.txt", "w");
    if (fp == NULL) return 1;

    fputs("Roses are red,\n", fp);
    fputs("Violets are blue.\n", fp);
```

```
    fclose(fp);
    printf("Poem written to file\n");
    return 0;
}
```

OUTPUT

```
Poem written to file
```

Advantages	Limitations
• Writes an entire string in a single call — faster and shorter code than looping with `putc()`. • Does not waste space storing a newline the programmer did not ask for.	• Does not automatically append `'\n'`, unlike console-only `puts()`. • Provides no formatting (numbers, variables) — `fprintf()` is needed for that.

Exam Tip

Since `fputs()` does **not** add `'\n'` automatically (unlike `puts()`, which does for console output), the programmer must explicitly include `'\n'` in the string when a new line is required.

FIG 5.2B — STRING I/O PROCESS (FGETS / FPUTS)



fgets() stops at '\n', at n-1 characters, or at EOF, and always null-terminates the result; fputs() stops at the source string's '\0' and writes no terminator to the file.

5(g) feof() — Detecting End-of-File

PURPOSE

`feof()` tests whether the **end-of-file indicator** has been set for the given stream, i.e., whether the last read operation attempted to read past the end of the file.

SYNTAX

```
int feof(FILE *fp);
```

Parameter	Meaning
`fp`	Pointer to the stream to be checked.
Return Value	Non-zero (true) if end-of-file has been reached; `0` (false) otherwise.

Working Principle

Every `FILE` structure maintains an internal end-of-file flag (part of its status flags, Section 4.2). This flag starts cleared. It is set only when a read function (such as `getc()` or `fgets()`) actually **attempts** to read beyond the last byte of the file and fails. `feof()` simply reports the current state of this flag — it does not itself perform any I/O.

Example — Using feof() to Read Until End of File

PROGRAM

```
#include <stdio.h>
int main() {
    FILE *fp;
    char ch;
    fp = fopen("chars.txt", "r");
    if (fp == NULL) return 1;

    while (!feof(fp)) {
        ch = getc(fp);
        if (!feof(fp))
            printf("%c", ch);
    }
    fclose(fp);
    return 0;
}
```

OUTPUT

```
ABC
```

Advantages	Limitations
• Provides a clear, named way to test whether a stream has reached its end. • Useful after a read loop to distinguish "ended normally" from a possible error (with `ferror()`).	• Only becomes true *after* a read has already failed — it cannot predict end-of-file in advance. • Using it as the sole loop condition before processing data is a classic source of bugs (see below).

Common Error — "while(!feof(fp))" Pitfall

A very common mistake is writing:

```
while (!feof(fp)) { ch = getc(fp); printf("%c", ch); }
```

This prints the **last character twice** (or a garbage character), because `feof()` only becomes true *after* a failed read attempt — not immediately when the last real character is read. The safer, exam-recommended pattern is:

```
while ((ch = getc(fp)) != EOF) { printf("%c", ch); }
```

FIG 5.3 — EOF DETECTION FLOWCHART



Recommended pattern: check the return value of getc()/fgets() directly against EOF/NULL rather than relying solely on feof() inside the loop condition.

Section 5 — Function Quick-Reference

Function	Purpose	Returns
`fopen()`	Open a file / associate a stream	`FILE*` or NULL
`putc()`	Write one character	Character written or EOF
`getc()`	Read one character	Character read or EOF
`fclose()`	Close a stream, flush buffer	0 or EOF
`fgets()`	Read a line/string	str or NULL
`fputs()`	Write a string	Non-negative or EOF
`feof()`	Test end-of-file indicator	Non-zero or 0

SECTION 6

Simple File-Based Programs Showing the Working of Different Members of FILE Structure

This section presents solved programs that demonstrate practical use of the functions covered in Section 5, in increasing order of complexity. Each program includes an explanation and expected output.

6.1 Program 1 — File Creation

Explanation: Demonstrates the most basic operation — creating an empty (or minimally populated) text file using `fopen()` in `"w"` mode.

PROGRAM

```
#include <stdio.h>
int main() {
    FILE *fp = fopen("student.txt", "w");
    if (fp == NULL) {
        printf("Unable to create file\n");
        return 1;
    }
    printf("File 'student.txt' created successfully\n");
    fclose(fp);
    return 0;
}
```

OUTPUT

```
File 'student.txt' created successfully
```

6.2 Program 2 — Writing Data to a File

Explanation: Uses `fputs()` to write multiple lines of student data into the file created above.

PROGRAM

```
#include <stdio.h>
int main() {
    FILE *fp = fopen("student.txt", "w");
    if (fp == NULL) return 1;

    fputs("Roll No: 101, Name: Asha\n", fp);
    fputs("Roll No: 102, Name: Ravi\n", fp);
    fclose(fp);
    printf("Records written to student.txt\n");
    return 0;
}
```

OUTPUT

```
Records written to student.txt
```

6.3 Program 3 — Reading Data from a File

Explanation: Opens the same file in read mode and prints its content to the console using `fgets()` inside a loop.

PROGRAM

```
#include <stdio.h>
int main() {
    FILE *fp = fopen("student.txt", "r");
    char buffer[100];
    if (fp == NULL) return 1;

    while (fgets(buffer, sizeof(buffer), fp) != NULL)
        printf("%s", buffer);

    fclose(fp);
    return 0;
}
```

OUTPUT

```
Roll No: 101, Name: Asha
Roll No: 102, Name: Ravi
```

6.4 Program 4 — Copying File Contents

Explanation: Reads character-by-character from a source file and writes the same characters into a destination file — a classic file-copy program using `getc()` and `putc()`.

PROGRAM

```
#include <stdio.h>
int main() {
    FILE *src, *dest;
    char ch;

    src  = fopen("student.txt", "r");
    dest = fopen("backup.txt", "w");
    if (src == NULL || dest == NULL) {
        printf("Error opening files\n");
        return 1;
    }

    while ((ch = getc(src)) != EOF)
        putc(ch, dest);

    fclose(src);
    fclose(dest);
    printf("File copied to backup.txt\n");
    return 0;
}
```

OUTPUT

```
File copied to backup.txt

(backup.txt now contains an exact copy of student.txt)
```

6.5 Program 5 — Character-by-Character File Operations

Explanation: Counts the number of characters present in a text file by reading one character at a time until `EOF` is returned.

PROGRAM

```
#include <stdio.h>
int main() {
    FILE *fp = fopen("student.txt", "r");
    char ch;
    long count = 0;
    if (fp == NULL) return 1;

    while ((ch = getc(fp)) != EOF)
        count++;

    fclose(fp);
    printf("Total characters: %ld\n", count);
    return 0;
}
```

OUTPUT

```
Total characters: 51
```

6.6 Program 6 — Line-by-Line File Operations

Explanation: Counts the number of lines in a text file by reading it line-by-line with `fgets()`.

PROGRAM

```
#include <stdio.h>
int main() {
    FILE *fp = fopen("student.txt", "r");
    char line[100];
    int lineCount = 0;
    if (fp == NULL) return 1;

    while (fgets(line, sizeof(line), fp) != NULL)
        lineCount++;

    fclose(fp);
    printf("Total lines: %d\n", lineCount);
    return 0;
}
```

OUTPUT

```
Total lines: 2
```

6.7 Program 7 — Reading Until EOF (feof-based check)

Explanation: Demonstrates the correct, exam-safe way of using `feof()` together with `getc()` to read a file until its end, avoiding the "double last character" pitfall discussed in Section 5(g).

PROGRAM

```
#include <stdio.h>
int main() {
    FILE *fp = fopen("student.txt", "r");
    char ch;
    if (fp == NULL) return 1;

    ch = getc(fp);
    while (!feof(fp)) {          /* check BEFORE using ch      */
        printf("%c", ch);
        ch = getc(fp);           /* read next, THEN test again */
    }
    fclose(fp);
    return 0;
}
```

OUTPUT

```
Roll No: 101, Name: Asha
Roll No: 102, Name: Ravi
```

FIG 6.1 — SUMMARY FLOW: TYPICAL FILE-BASED PROGRAM



Section 6 — Key Takeaways

- Every file program follows: declare pointer → open → validate → process → close.
- Character-level operations use `getc()` / `putc()`; line-level operations use `fgets()` / `fputs()`.
- File copying combines a read stream and a write stream simultaneously.
- Prefer checking the function's own return value (EOF/NULL) over `feof()` alone for loop termination.

SECTION 7

Quick Revision

7.1 Important Definitions (One-Line Recall)

Term	One-Line Definition
File	A named collection of data stored permanently on secondary storage.
Text File	A file storing human-readable, line-oriented character data.
FILE	A predefined structure type in stdio.h that holds control information for a stream.
File Pointer	A pointer (`FILE *`) that references the FILE structure of an opened stream.
Buffering	Temporary storage of data in memory to reduce the number of slow disk accesses.
Stream	A logical channel connecting a program to a file or device for I/O.
EOF	A special value (usually -1) returned by I/O functions to indicate end-of-file.

7.2 File Opening Modes — Master Table

Mode	Meaning	Read?	Write?	Creates New?	Truncates Existing?
`r`	Read	Yes	No	No	No
`w`	Write	No	Yes	Yes	Yes
`a`	Append	No	Yes (at end)	Yes	No
`r+`	Read/Write	Yes	Yes	No	No
`w+`	Write/Read	Yes	Yes	Yes	Yes
`a+`	Append/Read	Yes	Yes (at end)	Yes	No

Tip

Append "b" to any mode (e.g. `"rb"` , `"wb+"`) to open the file in binary mode instead of the default text mode.

Simple Way to Remember

- `"r"` = **read only** — file must already exist, nothing is ever erased or created.
- `"w"` = **wipe and write** — always starts from a blank file (old content destroyed if it existed).
- `"a"` = **add at the end** — old content preserved, new content tacked on after it.

7.3 Summary of File I/O Functions

Function	Prototype	Use
`fopen`	FILE *fopen(const char*, const char*)	Open a file / create a stream
`putc`	int putc(int, FILE*)	Write one character
`getc`	int getc(FILE*)	Read one character
`fclose`	int fclose(FILE*)	Close stream & flush buffer
`fgets`	char *fgets(char*, int, FILE*)	Read a line/string
`fputs`	int fputs(const char*, FILE*)	Write a string
`feof`	int feof(FILE*)	Test end-of-file indicator

7.4 Frequently Asked Exam Questions

Short Answer / One-Mark

1. Define a file. What is the need for file handling in C?
2. What is FILE in C? Where is it defined?
3. What does a file pointer store — data or an address?
4. Differentiate between text files and binary files. On what basis are they classified?
5. What is the return value of `fopen()` on failure?
6. Give the difference between `"w"` and `"a"` mode.

7. What does `feof()` return, and when?

Descriptive / Long Answer

1. Explain the concept of a file and the various file opening modes in C with examples.
2. Describe the internal (conceptual) structure of FILE and explain the purpose of its members.
3. What is buffering? Explain its role in file I/O with a diagram.
4. Write a C program to copy the contents of one file into another using `getc()` and `putc()`.
5. Write a C program to count the number of characters and lines in a text file.
6. Explain `fgets()` and `fputs()` with syntax, parameters, and a suitable example.
7. Explain why `while(!feof(fp))` can produce incorrect output; suggest the correct approach.

7.5 One-Page Revision Notes

Everything on One Page

- **File** = permanent, named data storage on disk; needed because RAM is volatile and limited.
- **Types (classified by data-representation format):** text files (readable, line-based) vs binary files (raw internal format).
- **FILE** = structure type in stdio.h; **FILE *fp** = pointer used to reference an open stream.
- **FILE structure members (conceptually):** file descriptor, buffer pointer, buffer size, current position pointer, status/EOF flags.
- **Buffering** reduces disk access by holding data temporarily in memory (fully buffered / line buffered / unbuffered).
- **Cycle:** declare `FILE *fp` → `fopen()` → check NULL → read/write → `fclose()`.
- **fopen(name, mode)** → returns FILE* or NULL.
- **putc(ch, fp)** writes one char; **getc(fp)** reads one char; loop until `EOF`.
- **fclose(fp)** flushes buffer to disk and releases the stream.
- **fgets(str, n, fp)** reads a line (keeps '\n'); **fputs(str, fp)** writes a string (does NOT add '\n').
- **feof(fp)** becomes true only after a read attempt goes past the last character — never use it as the sole loop condition before printing.

7.6 Common Mistakes and Debugging Tips

Mistake	Fix / Tip
Not checking `fopen()` for NULL	Always test `if (fp == NULL)` before using the pointer.
Using `char` instead of `int` for `getc()`'s return value	Always store the result of `getc()` / `fgetc()` in an `int` variable.
Using `while(!feof(fp))` as the only loop test	Test the return value of the read function itself, e.g. `while((ch=getc(fp))!=EOF)`.
Forgetting `fclose()`	Always close every opened stream to flush buffers and free resources.
Opening in wrong mode (e.g., "r" for a file that must be created)	Use "w"/"a" when the file may not yet exist; use "r" only when it must already exist.
Assuming `fputs()` adds a newline	Explicitly include `'\n'` in the string if a new line is required.
Mixing text mode and binary mode incorrectly	Use binary mode ("rb"/"wb") for non-text data such as structures/images.